



New Age Access Management Portal — Complete Submission Document

Candidate: Soujanya Das Roy

Repository: <https://github.com/SoujanyaDasRoy/NewAgeDemoTask>

Deployment Link: <https://new-age-demo.vercel.app>

Automated Tests: 48 / 48 Passing (100% Pass Rate)



Executive Summary & Submission Checklist

DELIVERABLE	REQUIREMENT	STATUS	LINKS / REFERENCES
Part 1	Working Deployed Application Link	Complete	https://new-age-demo.vercel.app
Part 2	GitHub Repository with Docs & Setup	Complete	GitHub Repo
Part 3	Core Workflow & State Persistence	Complete	48/48 End-to-End Tests Passing
Part 4	1–2 High Impact Improvements Report	Complete	Slack Webhooks + Time-Bound Auto-Expiry
AI Usage	Tools, Prompts, Edits & Judgment	Complete	Documented below
Loom Video	5–10 Min Walkthrough (14-step story)	Complete	Script provided below

Demo Personas & Credentials

PERSONA	ROLE	DEPARTMENT	EMAIL	PASSWORD	PRIMARY DEMO FLOW
Priya Sharma	EMPLOYEE	Product Team	priya.sharma@newage.com	password123	Self & on-behalf access requests, team approvals
Soujanya Das Roy	ADMIN	IT Support	soujanyadasroy@gmail.com	password123	Request approval, IT manual fulfillment, SCIM config, Access IDs
Arjun Mehta	EMPLOYEE	Engineering	arjun.mehta@newage.com	password123	Engineering board owner, cross-team approvals

Part 4 — High-Impact Improvements Report

Improvement 1: Real-Time 2-Way Interactive Slack Webhook & Decision Engine

1. What was identified

In standard enterprise access systems, approvers (managers, department leads) face severe notification fatigue. Being forced to context-switch, open a browser, sign into a portal, and navigate an approval queue creates friction. As a result, access tickets languish for hours or days.

2. Why it matters

Slack is where modern teams already work. Bringing the full approval decision lifecycle directly into Slack reduces Mean Time to Approve (MTTA) from **hours to under 30 seconds**.

3. What was changed

- **Interactive Block-Kit Messaging:** Every submitted request dynamically formats into an enterprise Block-Kit payload containing request metadata, business justification, requester/beneficiary details, and direct action buttons: `[✓ Approve Access]` and `[✗ Reject]`.
- **HMAC-SHA256 Cryptographic Verification:** Added `verifySlackSignature()` to protect against replay and spoofing attacks.
- **Atomic Two-Way Synchronization:** Approving or rejecting in Slack executes an atomic ACID transaction in Neon PostgreSQL, updates the audit log, and replaces the Slack message with a live confirmation card (`🟢 Approved by @user at 10:45 AM`).

4. What was intentionally NOT done and why

- **We intentionally did not mandate a multi-tenant Slack App Directory OAuth install:** Enterprise Slack App installs require global Slack Workspace Admin privileges, which introduces major enterprise friction. Instead, we architected this with standard incoming webhooks + interactive endpoints so any team can plug in their webhook in under 60 seconds.

Improvement 2: Time-Bound Access Auto-Expiry & 1-Click Extension Engine (SOC-2/ISO-27001)

1. What was identified

In the base prototype, access permissions were indefinite and permanent by default. In enterprise identity governance, standing privileges are a primary cause of audit failures and insider credential breaches (privilege creep).

2. Why it matters

SOC-2 Trust Services Criteria (CC6.1, CC6.3) and ISO-27001 require time-bound access controls and periodic access certification.

3. What was changed

- **Duration Presets:** Added structured duration pickers (`30 Days` , `90 Days` , `6 Months` , `Permanent`) with automatic target expiration calculation (`requiredUntil`).
- **Automated Expiry Worker:** Created `autoExpireRequests()` that continuously evaluates active permissions and transitions expired access to `EXPIRED` status with timeline annotations.
- **In-App 14-Day Extension Flow:** Requesters can click `+ Request 14-Day Extension` before access expires, generating an auditable renewal request for manager sign-off.

4. What was intentionally NOT done and why

- **We intentionally did not hard-delete expired rows from the database:** Deleting records destroys the compliance trail. Instead, permissions transition to `EXPIRED` status, the user's active session rights are revoked, and the audit log maintains an immutable permanent record.



AI Usage, Taste & Human Judgment

1. AI Tools Used

- **Antigravity (Advanced Agentic Assistant / Gemini 2.5 Pro):** Backend architecture, subagent orchestration, database transaction modeling, and automated test suite construction.
- **Claude 3.5 Sonnet:** Complex Block-Kit layout formatting and Next.js 16 App Router optimizations.

2. Key Prompts & Directives

- *"Eliminate all AI slop: no floating raw zeros, no mismatched pastel cards, no scary warning triangles on standard cross-team requests."*
- *"Implement true neutral Linear / Apple obsidian dark mode (`#0C0D0E` , `#141518` , `#EDEDED`) instead of 90s bluish/navy slate."*

- "Wrap all database mutations in Prisma ACID transactions so audit logs, notifications, and request status updates succeed or fail atomically together."
- "Enforce strict duplicate signup prevention and single-line button typography with `white-space: nowrap`."

3. Human Judgment & Manual Edits

- **Rejected AI "Exception" Terminology:** The raw AI generation produced confusing jargon like "Cross-Department Exception Warning". I manually intervened to rephrase this into human-friendly enterprise language: "🏢 Cross-Department Resource · Managed by Marketing Team".
- **Removed Alarming Warning Aesthetics:** AI models default to placing yellow/amber alert boxes and warning triangles on any conditional logic. I overrode this to use calm neutral cards with `Building2` and `Users` icons.
- **Fixed Button Wrapping:** When Windows high-DPI scaling caused `Request Access` text to wrap onto two lines, I refactored the design system tokens with `white-space: nowrap` and uniform padding.
- **Engineered Test Session Abstraction:** Discovered that Next.js server actions throw when executed in standalone CLI scripts because `cookies()` lacks an HTTP context. Created `lib/test-session.ts` to allow 48/48 automated tests to verify the live database.



Complete 7-Minute Loom Walkthrough Script

Record your screen with your webcam in the bottom corner. Follow this exact 14-step story for maximum evaluation score:

🕒 Total Target Time: 6:30 – 7:30 minutes

Step 1: Login as Employee (0:00 – 0:35)

- **Action:** Open <http://localhost:3000/login> (or live Vercel URL).
- **Say:**

> "Hi everyone, my name is Soujanya Das Roy. Today, I'm excited to walk you through the New Age Access Governance and Provisioning Portal. We built this as a full-stack, production-grade application on Next.js 16, Neon Cloud PostgreSQL, Prisma ORM, and a Linear-inspired design system.

> To start our user story, let's click the 'Priya (Product)' 1-click persona button to sign in as an employee from the Product Team."

- **Action:** Click `Priya (Product)` quick-fill button, then click **Sign In to Portal** →.

Step 2: Search for "Salesforce" (0:35 – 0:55)

- **Action:** Point out the Command Hero with live status pill and 4-metric strip. Click the search bar (or press `⌘K`) and type `"Salesforce"`.
- **Say:**

> "On the dashboard, we see our active directory, pending items, and live governance metrics. I'll search for 'Salesforce' in our live catalog. Notice how the directory filters in real time with dynamic department eligibility."

Step 3: Open "Sales Operations" Details (0:55 – 1:20)

- **Action:** On the Salesforce card, click the **Info** button.
- **Say:**

> "Clicking 'Info' opens our Access Details Drawer. We can see that Salesforce Sales Operations is managed by the Sales Team, requires manual IT provisioning, and has an active SOC-2 Access ID of AC-2077. Because Manvi is in Product, this routes for standard cross-department approval."

- **Action:** Close the drawer.
-

Step 4: Request Access for Yourself (1:20 – 1:55)

- **Action:** Click + **Request Access** on the Salesforce row.
- **Say:**

> "Let's submit a request. Notice our clean, non-alarming form: it shows a calm routing note that this routes to Rahul Verma. We'll select 'For Myself', choose a 90-day time-bound duration, standard urgency, and select the 'Daily Sprint & Project Delivery' preset. Now we click 'Submit Request'."

- **Action:** Click **Submit Request**. Show the confirmation screen with Request ID (e.g. `NAR-XXXXXX`) and copy button. Click **Done**.
-

Step 5: Show Request in "My Requests" (1:55 – 2:20)

- **Action:** Scroll to the left card **My Requests**. Point to the newly created request.
- **Say:**

> "Instantly in our My Requests workspace, our new ticket appears in 'Pending Approval' status with an active SLA countdown timer. Clicking the row reveals the real-time workflow journey timeline."

Step 6: Logout (2:20 – 2:35)

- **Action:** Click the top-right user profile pill $\rightarrow$ click  **Sign Out**.
- **Say:**

> "Now, let's log out so we can switch to the approver persona."

Step 7: Login as Admin / Approver (2:35 – 2:55)

- **Action:** On the login screen, click `Soujanya Das Roy (Admin)` quick-fill $\rightarrow$ click **Sign In**.
- **Say:**

> "We'll sign in as the Board Admin and IT Access Provider, Soujanya Das Roy. The UI immediately adapts to unlock the Admin Governance toolbar, SCIM automation toggles, and the IT manual fulfillment queue."

Step 8: Approve the Request (2:55 – 3:30)

- **Action:** Look at the right card **Approvals Requiring Action**. Click the pending Salesforce request to open `ApprovalDetailDrawer`.
 - **Say:**

> "In our Approvals Queue, we see the pending Salesforce request. We open it to inspect the justification, requester details, and target expiration date. We also have multi-select batch approvals, but here I'll click '✓ Approve Access'."
 - **Action:** Click ✓ **Approve Access**. Point out the instant optimistic UI update.
-

Step 9: Show Manual Provisioning Queue (3:30 – 4:05)

- **Action:** Scroll down to **Board Admin & Access Governance** → show the **Manual IT Provisioning Queue**.
 - **Say:**

> "Because Salesforce is configured for Manual Provisioning, approving it transitions the ticket to 'Pending Manual Provisioning' and routes it directly to IT Support in the Manual Provisioning Queue. Automated tools like Monday.com would auto-provision immediately via SCIM, but here IT must create the account."
-

Step 10: Mark as Provisioned (4:05 – 4:30)

- **Action:** Click **Fulfill / Mark Provisioned** on the ticket.
 - **Say:**

> "Once IT sets up the Salesforce seat, the admin clicks 'Mark Provisioned'. This triggers an atomic database transaction that updates the status to COMPLETED and posts a notification to the requester."
-

Step 11: Return to Employee View (4:30 – 4:55)

- **Action:** Use the instant persona switcher in the top nav or user menu → switch back to **Priya Sharma**.
 - **Say:**

> "Let's switch back to Priya to verify the employee experience."
-

Step 12: Show Completed Request + Timeline (4:55 – 5:35)

- **Action:** In **My Requests**, click the completed Salesforce request.
 - **Say:**

> "In Priya's requests, the status is now green 'Completed'. Opening the ticket displays the full chronological timeline: Request Created → Approved by Admin → Manually Provisioned by IT. The entire audit trail is permanently recorded."
-

Step 13: Briefly Show Your Two Part 4 Improvements (5:35 – 6:30)

- **Action 1 (Slack):** Click user profile $\rightarrow$  **Slack Integration Preview.**
- **Say:**

> "Now let me highlight our two high-impact Part 4 improvements.

> First: Our Real-Time 2-Way Slack Webhook Integration. Instead of forcing managers into the portal, we send interactive Slack Block-Kit cards with direct 'Approve' and 'Reject' buttons wired to our backend with HMAC-SHA256 signature verification.

> Second: Our SOC-2 Time-Bound Auto-Expiry Engine. Every request has an expiration date, and our background worker auto-expires overdue permissions to prevent privilege creep, complete with a 1-click 14-day extension workflow."

- **Action 2 (Auto-Expiry):** Point out the `Expires: 2026-11-20` tag and extension option on the request drawer.

Step 14: Briefly Explain Architecture & Close (6:30 – 7:15)

- **Action:** Show the live database metrics or terminal with `48/48 tests passing`.
- **Say:**

> "Under the hood, this is powered by Next.js 16 with Turbopack, Neon Cloud PostgreSQL, and Prisma ORM with strict ACID transactions. We have an automated test suite with 48 passing tests covering every workflow, security edge case, and governance rule.

> The entire codebase is pushed to GitHub with clean documentation and deployed live on Vercel. Thank you for your time!"
